

Final report TTK4145

Group 51

April 2025

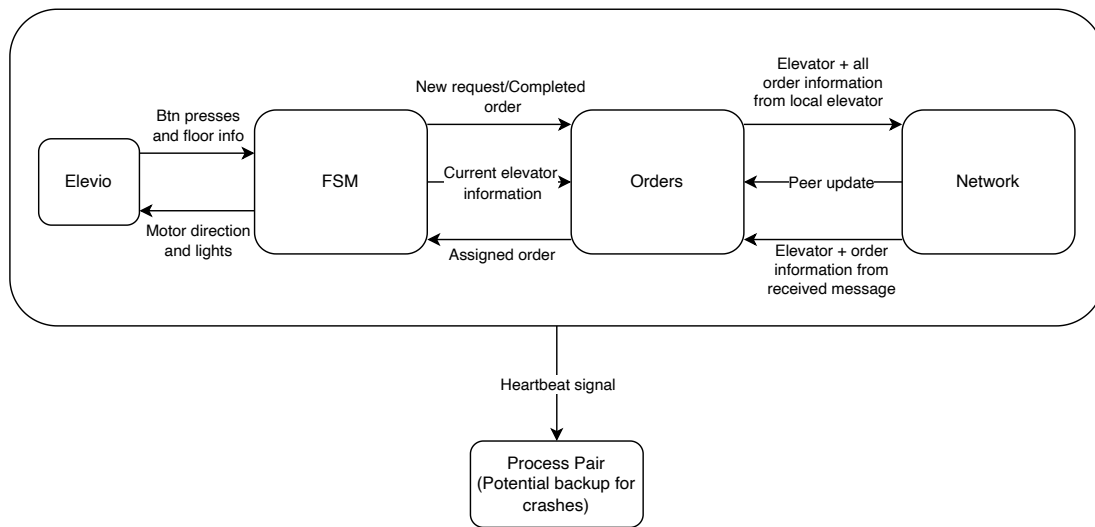


Figure 1: Overview of the modules of the system, showing how they interact with each other.

1 Design description

1.1 Network

The network module is visualized in Figure 2. The elevators communicate over the network by continuously sending an `ElevatorStateMsg`, containing their ID, all states and the order overview in the form of cyclic counter values in the map `AssignedOrders`.

The message is broadcasted to all elevators on the network, using UDP, every 10 milliseconds. These frequent updates ensure every elevator maintains up-to-date information about the others, while also acting as a heartbeat to detect connections and disconnections even with packet loss. If messages from an elevator are not detected for 0.5 seconds, each elevator removes that corresponding id from their network overview (peer list), assuming it has gone offline.

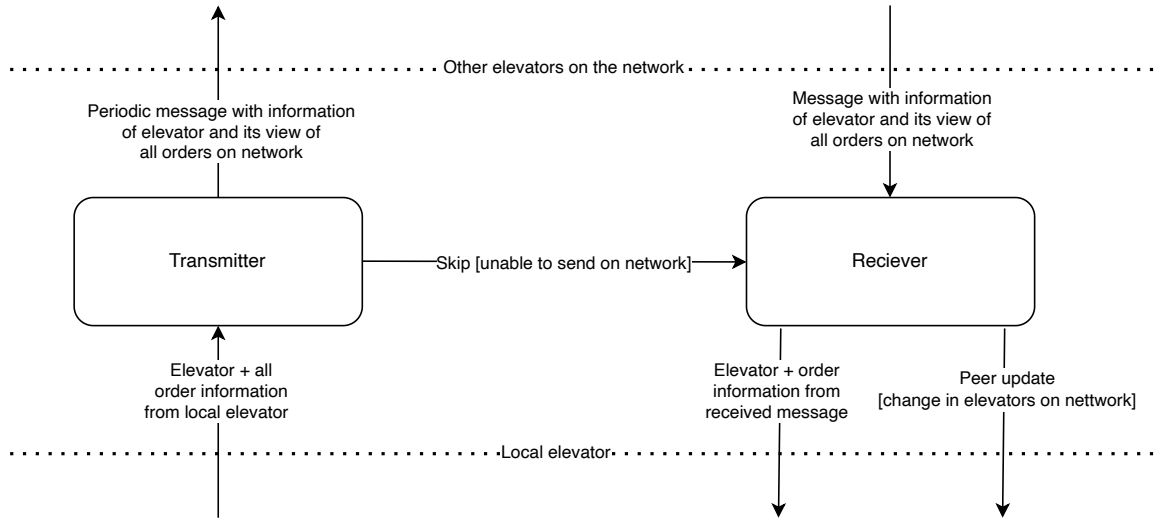


Figure 2: Overview of the network modules two main functions and how they work.

If network transmission is not possible, the transmitter has a fallback mechanism, seen in figure 2, which bypasses the usual UDP message transmission. This direct transmission prevents other modules from being delayed or stalled when UDP message sending fails, ensuring continued operation and minimizing potential disruptions. The reason it was done this specific way was to keep the peer list functionality operational.

1.2 Orders

Orders handles the merging of connected elevators into the system and the assignment and reassignment of requests/orders to an elevator on the network.

Cab and hall orders for all elevators (connected and disconnected) is kept in a map `assignedOrders`, with elevators as entries. Each entry corresponds to the last known state of the orders for a given elevator, in the form of a cyclic counter. This ensures correct synchronization of data between different modules.

When an update from another elevator is received through the network module, the `assignedOrders` maps of the local elevator and the external are merged for each entry. Information about the states of the external elevator is also saved. This ensures that all elevators on the network have up to date information about the others and the current state of orders.

Since the orders of a disconnected elevator are kept on the network, information about any unfinished cab orders will be given by the merging of data when it reconnects, happening after a

peerUpdate. This ensures that any cab orders in progress are not lost. If the elevator was only disconnected from the network (no crash) and completed the cab orders, it ignores the information about cab calls from other elevators.

The hall orders of malfunctioning or disconnected elevators are handled by other elevators on the network. These orders are reassigned to working elevators, ensuring that no hall orders are lost as long as there is at least one working elevator on the network. An example of the reassignment of a hall order can be seen in figure 4

Orders, a result of a button press or reassignment of orders, are done by an elevator and are either assigned to itself or others. Cab orders can only be assigned to itself, while hall orders use a cost function to assign orders, making the system behave sensibly and mostly efficiently. A sequence diagram, e.g. showing a cab order assignment can be seen in figure 3. An order is only to be started and sent to the FSM when all elevators on the network have the order registered in their assignedOrders map.

Hall lights are turned on if any entry in the assignedOrders map have an order.

1.3 FSM

The FSM module is in principle working as a single elevator. The order module acts as an interface for the FSM, giving orders and receiving information about button presses and completed orders. Because of this the FSM can operate independently of the order modules outside of the new orders given. Therefore all orders given to the FSM will be completed, as long as there are no errors (motor stoppage, obstruction or system crash) occurring.

To detect motor stoppage, a dedicated goroutine was implemented, that detects motor stop if an elevator uses more than 5 seconds traveling between two floors. A similar goroutine was also implemented for the door open timer. This timer was reset every time the door should be opened/keep opened and in the event of an obstruction the timer was continuously reset. To keep track of motorstop and obstruction of an elevator, flags for either one of these were used.

Cab lights are activated if the FSM have any active cab orders. Door lights are active as long as the door timer is active.

1.4 Elevio and Process Pair

Elevio is the same module as given from source resources, being the interaction between the physical i/o and the elevator system.

Process Pair is a potential backup system spawning another program listening to heartbeats (different than the one in the network module) from the original and taking over if the original crashes. This module is by default disabled, but may be used if wanted.

2 Case study of important decisions

2.1 Cyclic counter implementation

The implementation of the cyclic counter was the most time-consuming and confusing part of the process, mainly due to our lack of theoretical understanding. To function correctly, the counter required 'state checkpoints' that all elevators had to reach before proceeding. Initially, we struggled to determine what states to use and which to set as these checkpoints, and it was not before after revision of the theory we overcame this.

- **Alternative that was not chosen:** 'Order Confirmed' as the top state, requiring all elevators to synchronize at this point before completing their tasks.
- **Alternative that was chosen:** 'Order Complete' as the top state in the cyclic counter and an extra 'unknown' counter value. This ensured that each elevator had to confirm an order as completed before the counter could reset.

The first approach didn't work because the elevators ended up never clearing the orders. This was due to the order being cleared by an elevator before the other elevators could react. We did not realize that the elevator completing an order would reset the counter before other elevators could acknowledge the completion. This was a misunderstanding of the cyclic counter theory.

The alternative that was chosen made the elevator states sync without issue, by only resetting the cyclic counter when all elevators were at 'Order Complete'.

The 'unknown' counter value was implemented to always be overwritten when merging world-views. When an elevator disconnects, the system could reassign its hall orders and then set them to 'unknown' in their own AssignedOrders map. This made it easy to merge the disconnected elevator into the network if it reconnects in the future, and be used as an initial value when starting an elevator for the first time.

2.2 Module interaction and data ownership

The modules and their interaction from the preliminary design were cluttered and had many non-intuitive interactions. This complexity made it difficult to implement the system, plan around the specifications, and ensure correct data ownership. We therefore decided to simplify the system into only containing the modules as shown in figure 1.

- **Alternative that was not chosen:** Adding a data module, which was responsible for maintaining and changing the data. This option has its advantages with only one module being responsible for data and therefore ensuring no concurrency problems related to sharing of data
- **Alternative that was chosen:** To let each module be responsible for its own data and send copies of this to the other modules.

The choice taken was partially due to its simpler structure and more intuitive interaction between modules, but also that it would fit nicer with the existing code, needing less work for implementation. Using this solution we still ensured correct data ownership, with each module being responsible for its own data and passing copies of the data to the other modules for them to use. An argument could be made for using the first option, with this more or less guaranteeing correct data ownership. The first option might be a better solution, but should in that case been implemented earlier in the process, reducing need of restructuring of functions and their functionality.

From this we learned that planning ahead is key, and spending more time on the design process is better than rethinking during the implementation process. Related to programming we learned how to ensure correct data ownership with message passing. We learned that in go lang some datatypes requires the implementation of a deepcopy to ensure that data is passed correctly between modules.

As for improvements we absolutely acknowledg that our module design/what each modules include are not perfect, with especially the orders module being too big. If we were to do this project again we would try to keep this module more under control, preferably splitting it into multiple submodules.

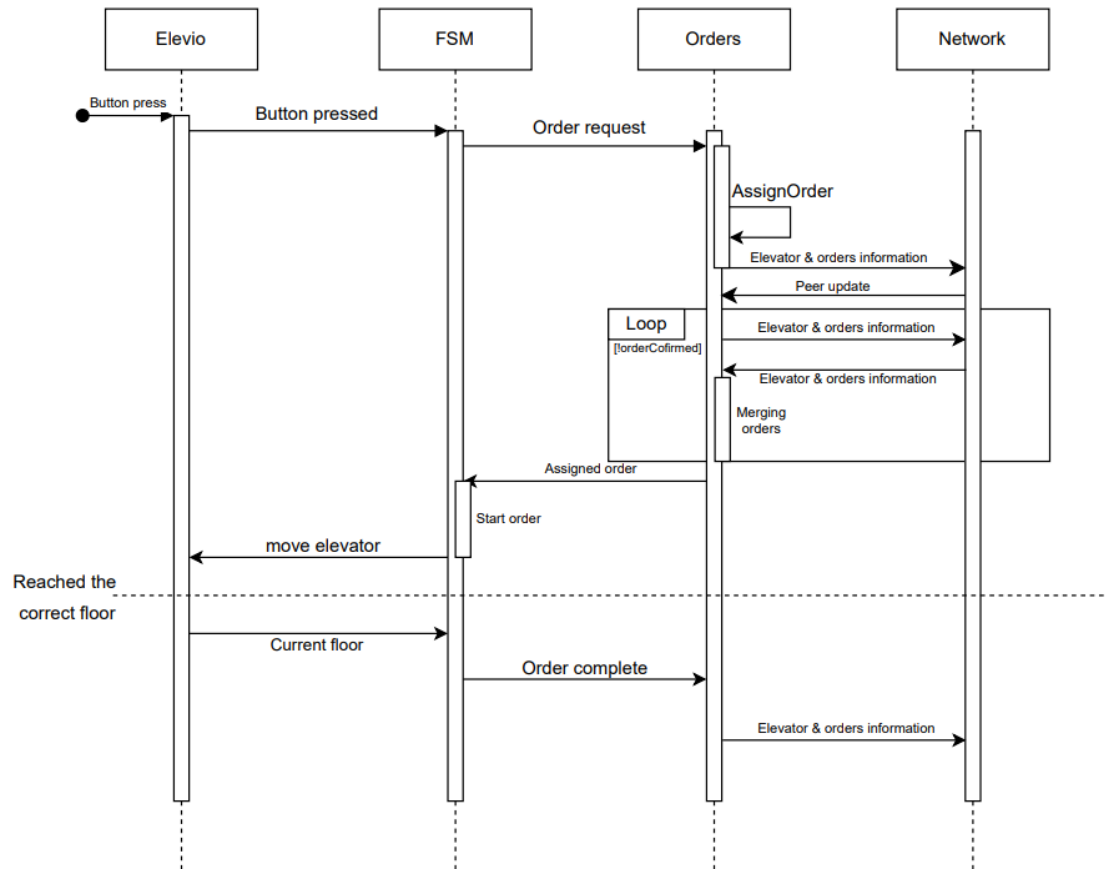


Figure 3: Order sequence diagram for a normal order event. The order corresponding to the button press is assigned to the same elevator.

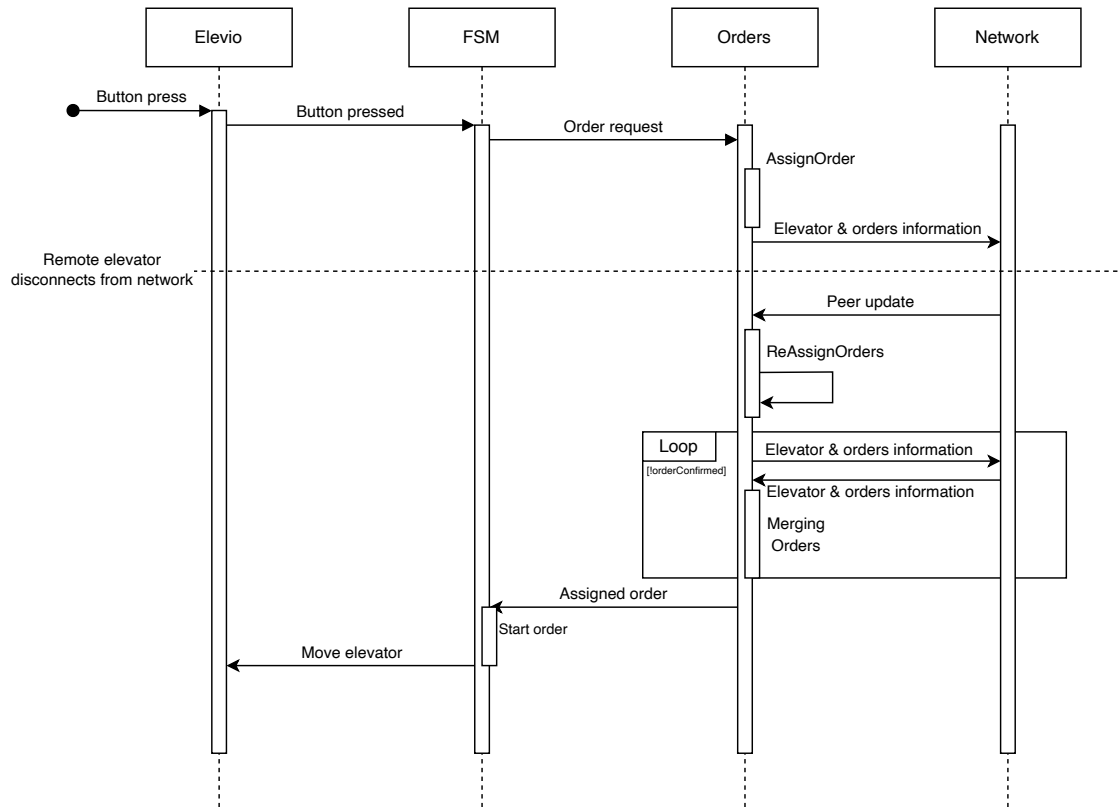


Figure 4: Order sequence diagram for an order event disrupted by network disconnection. The order corresponding to the button press is first assigned to an elevator on the network, but later reassigned to the elevator where the button press happened.